

SONiX 32-Bit Cortex-M0 Micro-Controller

DMA SPI Example Code

SONiX reserves the right to make change without further notice to any products herein to improve reliability, function or design. SONiX does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights nor the rights of others. SONiX products are not designed, intended, or authorized for use as components in systems intended, for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SONiX product could create a situation where personal injury or death may occur. Should Buyer purchase or use SONiX products for any such unintended or unauthorized application. Buyer shall indemnify and hold SONiX and its officers, employees, subsidiaries, affiliates and distributors harmless against all claims, cost, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use even if such claim alleges that SONiX was negligent regarding the design or manufacture of the part.



AMENDENT HISTORY

Version	Date	Description
1.0	2024/11/27	1. First version.

Table of Content

AMENDENT HISTORY	2
1 OVERVIEW	4
2 FEATURE	4
3 HW	4
4 EXAMPLE CODE	5
4.1 INITIALIZATION	5
4.2 DMA SPI FUNCTION ENABLE	6
5 RESULT	7

1 OVERVIEW

This document provides our customers the C program examples of SPI in DMA mode.

2 FEATURE

- DMA SPI TX mode
- DMA SPI RX mode
- SPI DMA Full-Duplex Mode
- Use ARM-compiler V6.12

3 HW

- The starter kit board of vendor desired MCU.
- Connect MOSI0 to MISO0.

4 EXAMPLE CODE

4.1 Initialization

Initial SPI and DMA function. User can modify the settings of SPI and DMA in these function. The demo code demonstrates SPI DMA full-duplex mode. DMA use 2 channel to achieve SPI full-duplex function. When RX FIFO is greater than the threshold, DMA request is issued for channel 0. When TX FIFO is less than the threshold, DMA request is issued for channel 1.

If user want to use DMA SPI as only TX or RX mode, just call the DMA SPI TX or RX related functions only.

```

78 // Initial SPI
79 SPI0_Init();
80
81 //Auto-SEL enable
82 SN_SPI0->CTRL0_b.SELDIS = SPI_SELDIS_EN;
83
84 // Disable SPI ISR when using DMA SPI
85 NVIC_DisableIRQ(SPI0_IRQn);
86
87 // Init DMA
88 DMASPI0TXInit();
89 DMASPI0RXInit();

```

```

131 void DMASPI0TXInit(void)
132 {
133     DMA_InitSt stDMACH_Init;
134
135     // The settings of SPI FIFO TX/RX TH and DMA width/burst
136     // Data length = 8bit, SPI FIFO depth = 16 bytes
137     SN_SPI0->FIFO_TH_b.NEW_TH_EN = SPI_NEW_TH_EN;
138     SN_SPI0->FIFO_TH_b.NEW_TXFIFO_TH = SPI_NEW_TXFIFOTH_4;
139
140     // Set DMA CH0 as M(SRAM) to P(SPI TX) mode
141     // Source : Memory, Increment, 8bit, no request, Burst 4
142     stDMACH_Init.b_SrcAddrCtrl = DMA_SRCAD_CTL_INC;
143     stDMACH_Init.b_SrcMode = DMA_SRC_MEMORY;
144     stDMACH_Init.b_SrcWidth = DMA_SRC_WIDTH_8BIT;
145     stDMACH_Init.b_SrcReqSel = DMA_SRCRS_NONE;
146     stDMACH_Init.b_BurstSize = DMA_BURST_4;
147     // Destination : Peripheral SPI, Fixed, 8bit, SPI0 TX
148     stDMACH_Init.b_DstAddrCtrl = DMA_DSTAD_CTL_FIX;
149     stDMACH_Init.b_DstMode = DMA_DST_PERIPHERAL;
150     stDMACH_Init.b_DstWidth = DMA_DST_WIDTH_8BIT;
151     stDMACH_Init.b_DstReqSel = DMA_DSTRS_SPI0_TX;
152     // Channel : LV0, Enable TC/ABT interrupt
153     stDMACH_Init.b_Priority = DMA_CHPRI_LV0;
154     stDMACH_Init.b_IntTCEn = DMA_INT_TC_MSK_DIS;
155     stDMACH_Init.b_IntABTEn = DMA_INT_ABT_MSK_DIS;
156
157     // Initial DMA
158     DMA_Init(&stDMACH_Init, eDMA_CH0);
159 }

```

```

169 void DMASPI0RXInit(void)
170 {
171     DMA_InitSt stDMACH_Init;
172
173     // The settings of SPI FIFO TX/RX TH and DMA width/burst
174     // Data length = 8bit, SPI FIFO depth = 16 bytes
175     SN_SPI0->FIFO_TH_b.NEW_TH_EN = SPI_NEW_TH_EN;
176     SN_SPI0->FIFO_TH_b.NEW_RXFIFO_TH = SPI_NEW_RXFIFOTH_3;
177
178     // Set DMA CH1 as P(SPI RX) to M(SRAM) mode
179     // Source : Peripheral SPI, Fixed, 8bit, SPI0 RX, Burst 4
180     stDMACH_Init.b_SrcAddrCtrl = DMA_SRCAD_CTL_FIX;
181     stDMACH_Init.b_SrcMode = DMA_SRC_PERIPHERAL;
182     stDMACH_Init.b_SrcWidth = DMA_SRC_WIDTH_8BIT;
183     stDMACH_Init.b_SrcReqSel = DMA_SRCRS_SPI0_RX;
184     stDMACH_Init.b_BurstSize = DMA_BURST_4;
185     // Destination : Memory, Increment, 8bit, no request
186     stDMACH_Init.b_DstAddrCtrl = DMA_DSTAD_CTL_INC;
187     stDMACH_Init.b_DstMode = DMA_DST_MEMORY;
188     stDMACH_Init.b_DstWidth = DMA_DST_WIDTH_8BIT;
189     stDMACH_Init.b_DstReqSel = DMA_DSTRS_NONE;
190     // Channel : LV0, Enable TC/ABT interrupt
191     stDMACH_Init.b_Priority = DMA_CHPRI_LV0;
192     stDMACH_Init.b_IntTCEn = DMA_INT_TC_MSK_DIS;
193     stDMACH_Init.b_IntABTEn = DMA_INT_ABT_MSK_DIS;
194
195     // Initial DMA
196     DMA_Init(&stDMACH_Init, eDMA_CH1);
197 }

```

SPI will transmit 256 bytes of data from 0x00 to 0xFF.

```

91 // Init data buffer
92 for(i = 0; i<DMA_TEST_SIZE ;i++)
93 {
94     // Set Tx data = 0x00, 0x01, ... ,0xFF
95     b_arySPITXData[i] = i & 0xFF;
96     // Set Rx data = 0x00
97     b_arySPIRXData[i] = 0x00;
98 }

```

4.2 DMA SPI function enable

SPI transmission must wait for the TX buffer to be written in SPI master mode. The SEL state will be kept 0 until all transmissions are completed after the transmission starts.

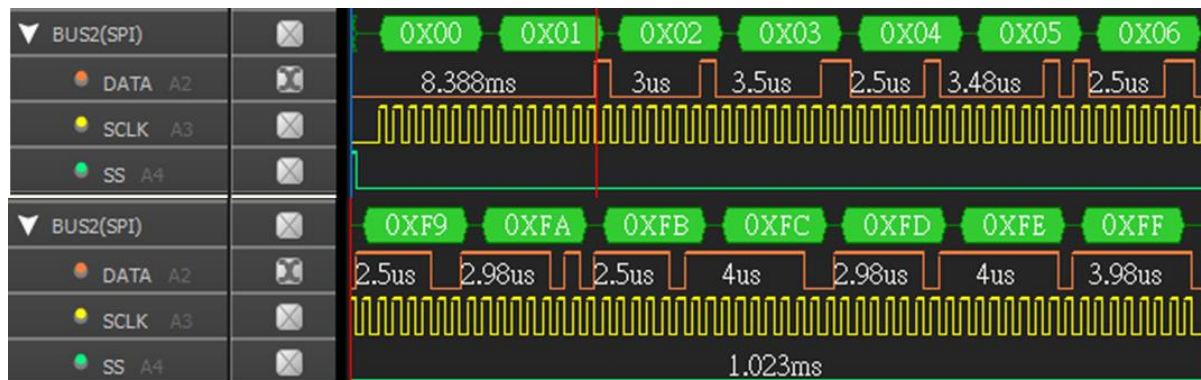
```
100 // Start DMA SPI TX/RX transfer
101 DMASPI0RXStart();
102 DMASPI0TXStart();
103 // Set SPI DMA Enable
104 SPI0_DMAEnable(mskSPI_TX_DMA_EN | mskSPI_RX_DMA_EN, DMA_TEST_SIZE);
```

The DMA TC flag and interrupt will be set when all data transfers are completed. The SPI TXEMPIF flag will be set after all data are sent in TX FIFO.

```
106 // Wait for DMA TC interrupt flag
107 while(stDMA0IntFlag[eDMA_CH0].Flag.bits.TC != 1);
108 while(stDMA0IntFlag[eDMA_CH1].Flag.bits.TC != 1);
109
110 // make sure all data has been TX though SPI
111 while(SN_SPI0->RIS_b.TXEMPIF != 1);
```

5 RESULT

The data sent or received can be observed on SPI bus.



The results can be observed through Watch 1 window or Memory window.

Memory 1	Memory 2
Address: b_arrySPITXData	Address: b_arrySPIRXData
0x2000000C: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F	0x20000010C: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
0x20000001C: 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F	0x20000011C: 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
0x20000002C: 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F	0x20000012C: 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F
0x20000003C: 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F	0x20000013C: 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
0x20000004C: 40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F	0x20000014C: 40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F
0x20000005C: 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F	0x20000015C: 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
0x20000006C: 60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F	0x20000016C: 60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F
0x20000007C: 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F	0x20000017C: 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
0x20000008C: 80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F	0x20000018C: 80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F
0x20000009C: 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F	0x20000019C: 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
0x2000000AC: A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF	0x2000001AC: A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF
0x2000000BC: B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF	0x2000001BC: B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
0x2000000CC: C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF	0x2000001CC: C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF
0x2000000DC: D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF	0x2000001DC: D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
0x2000000EC: E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF	0x2000001EC: E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF
0x2000000FC: F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF	0x2000001FC: F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF

SONIX reserves the right to make change without further notice to any products herein to improve reliability, function or design. SONIX does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights nor the rights of others. SONIX products are not designed, intended, or authorized for use as components in systems intended, for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SONIX product could create a situation where personal injury or death may occur. Should Buyer purchase or use SONIX products for any such unintended or unauthorized application. Buyer shall indemnify and hold SONIX and its officers, employees, subsidiaries, affiliates and distributors harmless against all claims, cost, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use even if such claim alleges that SONIX was negligent regarding the design or manufacture of the part.

Main Office:

Address: 10F-1, NO. 36, Taiyuan Street, Chupei City, Hsinchu, Taiwan R.O.C.
Tel: 886-3-5600 888
Fax: 886-3-5600 889

Taipei Office:

Address: 15F-2, NO. 171, Song Ted Road, Taipei, Taiwan R.O.C.
Tel: 886-2-2759 1980
Fax: 886-2-2759 8180

Hong Kong Office:

Unit No.705, Level 7 Tower 1, Grand Central Plaza 138 Shatin Rural Committee Road, Shatin, New Territories, Hong Kong.
Tel: 852-2723-8086
Fax: 852-2723-9179

Technical Support by Email:

Sn8fae@sonix.com.tw